
Lab : Kubernetes HPA + Load Testing + Monitoring

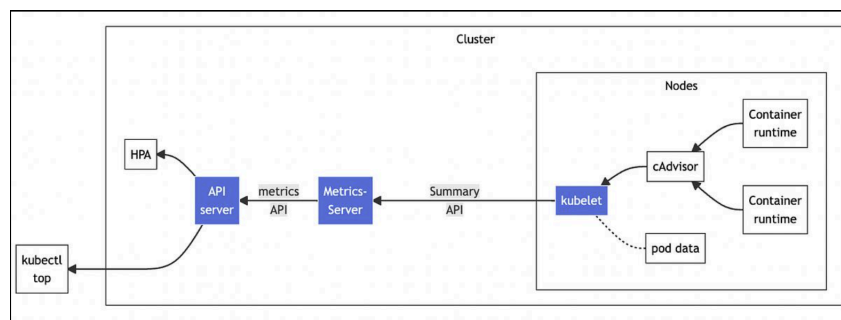
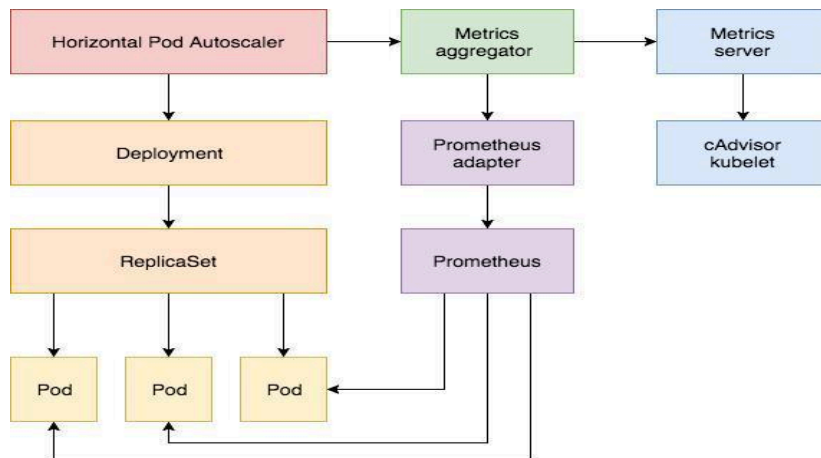
Table of Contents

1 Objectifs.....	2
2 Architecture globale.....	2
3 Structure du repo.....	3
4 Étape 1 — le cluster.....	3
4.1 Créer un nouveau ou utiliser un cluster existant.....	3
4.2 Créer un nouveau namespace, dédié à ce lab.....	4
4.3 se positionner sur le namespace du lab.....	4
5 Étape 2 — metrics-server.....	4
6 Étape 3 — Déployer l'application CPU.....	5
 app/deployment.yaml	5
 app/service.yaml	6
7 Étape 4 — Configurer le HPA.....	7
 hpa/hpa.yaml	7
8 Étape 5 — Prometheus + Grafana.....	8
8.1 Installation (si nécessaire).....	8
9 Étape 6 – Accéder à Prometheus.....	9
Login Prometheus	9
10 Étape 7 — Accéder à Grafana.....	9
Login Grafana	10
11 Vérifier le scaling HPA.....	10
12 Étape 8 — Load testing.....	11
12.1 test hey.....	11
12.2 — Observer.....	12
13 – verifier le scale down.....	14

1 Objectifs

- Déployer une app :
 - Activer **Horizontal Pod Autoscaler (HPA)**
 - Générer de la charge
 - Visualiser :
 - CPU
 - Pods
 - Scaling en temps réel dans Grafana
-

2 Architecture globale



3 Structure du repo

```
k8s-hpa-observability-lab/
├── app/
│   ├── deployment.yaml
│   └── service.yaml
├── hpa/
│   └── hpa.yaml
├── monitoring/
│   └── prometheus-grafana.yaml
└── README.md
```

4 Étape 1 — le cluster

👉 Exemple avec k3d :

4.1 Créer un nouveau ou utiliser un cluster existant.

```
k3d cluster create hpa-lab
kubectl get nodes
```

ou bien dans notre exemple utilisez le cluster préexistant qui sert à tous nos labs : platform-lab

On vérifiera en premier la liste des clusters existants , avec la commande suivante :

```
kubectl cluster list
```

```
henri@Henri:~/labs$ k3d cluster list
NAME          SERVERS  AGENTS  LOADBALANCER
gitops-lab    0/1      0/0     true
henri         0/1      0/0     true
ingress-lab   0/1      0/0     true
platform-lab  1/1      0/0     true
henri@Henri:~/labs$
```

On choisit le cluster : platform-lab (on vérifie qu'il est démarré. Ce qui est le cas car on voit 1/1 pour le serveur de ce cluster. S'il n'est pas démarré , le démarrer avec la commande : k3d cluster start platform-lab.

A la fin de l'exercice , on fera : k3d cluster stop platform-lab

4.2 Créer un nouveau namespace, dédié à ce lab

```
kubectl create namespace lab14-1
```

4.3 se positionner sur le namespace du lab

Il est utile de vérifier qu'on est sur le namespace du lab en cours, le namespace que l'on vient de créer.

Ce surtout quand on fait plusieurs labs sur la même machine , ou bien quand on bascule d'un environnement de travail vers un autre.

```
kubectl config view --minify | grep namespace
```

```
henri@Henri:~/labs$  
henri@Henri:~/labs$ kubectl config view --minify | grep namespace  
    namespace: lab16-1  
henri@Henri:~/labs$  
henri@Henri:~/labs$
```

Ici, par exemple , comme j'avais travaillé sur un autre lab, précédemment , au niveau du namespace lab16-1, le namespace actuel est resté lab16-1 et non pas celui du lab actuel. Changeons alors de namespace :

On se positionne sur le bon contexte (cluster) et aussi sur le namespace choisi

```
kubectl config set-context --current --namespace=lab14-1
```

```
henri@Henri:~/labs$  
henri@Henri:~/labs$ kubectl config set-context --current --namespace=lab14-1  
Context "k3d-platform-lab" modified.  
henri@Henri:~/labs$  
henri@Henri:~/labs$
```

5 Étape 2 — metrics-server

Vérifier si le serveur a été installé précédemment (autre lab) ou autre raison
Saisir la commande suivante : `kubectl get pods -n kube-system | grep metrics`

```
henri@Henri:~/labs$  
henri@Henri:~/labs$ kubectl get pods -n kube-system | grep metrics  
metrics-server-5985cbc9d7-j6xws          1/1      Running    0          6h16m  
henri@Henri:~/labs$
```

Dans notre cas , on voit que le serveur est déjà installé. On voit metrics-server qui est installé sur kube-system

Au cas ou le serveur de métriques ne serait pas installé, alors l'installer avec la commande suivante :

```
kubectl apply -f  
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml  
|
```

👉 Vérifier : On vérifie au final, avec la commande suivante , `kubectl top nodes`, qui montre la métrique du %CPU utilisée par le cluster et la métrique de mémoire utilisée

`kubectl top nodes`

```
henri@Henri:~/labs$  
henri@Henri:~/labs$ kubectl top nodes  
NAME                                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)  
k3d-platform-lab-server-0          263m         1%       2535Mi          18%  
henri@Henri:~/labs$  
henri@Henri:~/labs$  
henri@Henri:~/labs$
```

6 Étape 3 — Déployer l'application CPU

 **app/deployment.yaml**

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: cpu-app  
spec:  
  replicas: 1  
  selector:
```

```
matchLabels:
  app: cpu-app
template:
  metadata:
    labels:
      app: cpu-app
  spec:
    containers:
      - name: cpu-app
        image: vish/stress
        args:
          - "-cpus"
          - "1"
        resources:
          requests:
            cpu: "100m"
          limits:
            cpu: "500m"
```

 app/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: cpu-app
spec:
  selector:
    app: cpu-app
  ports:
    - port: 80
      targetPort: 8080
```

👉 Appliquer :

kubectl apply -f app/

```
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl apply -f app/
deployment.apps/cpu-app created
service/cpu-app created
henri@Henri:~/labs/k8s-hpa-observability-lab$
henri@Henri:~/labs/k8s-hpa-observability-lab$
```

Vérifier :

On a 1 pod et 1 application qui tourne sur ce pod

```
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl get svc
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)    AGE
cpu-app      ClusterIP   10.43.49.200    <none>       80/TCP     17s
henri@Henri:~/labs/k8s-hpa-observability-lab$
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
cpu-app-6fbc458fdb-vx9hx            1/1     Running   0          115s
henri@Henri:~/labs/k8s-hpa-observability-lab$
```

7 Étape 4 — Configurer le HPA

hpa/hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: cpu-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: cpu-app
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

👉 Appliquer :

kubectl apply -f hpa/

```
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl apply -f hpa/
horizontalpodautoscaler.autoscaling/cpu-app-hpa created
henri@Henri:~/labs/k8s-hpa-observability-lab$
henri@Henri:~/labs/k8s-hpa-observability-lab$
```

👉 Vérifier :

kubectl get hpa

```
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl get hpa
NAME          REFERENCE          TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
cpu-app-hpa   Deployment/cpu-app  cpu: 580%/50%    1         10        10         77s
henri@Henri:~/labs/k8s-hpa-observability-lab$
henri@Henri:~/labs/k8s-hpa-observability-lab$
```

8 Étape 5 — Prometheus + Grafana

8.1 Installation (si nécessaire)

Si pas déjà installé alors procéder à l'installation de prometheus et grafana. Si déjà installé précédemment et lors d'autres utilisations alors sauter au paragraphe 9 , suivant.

Si prometheus et grafana non encore installés , alors procéder à l'installation. Dans notre lab, ces utilitaires avaient déjà été ajoutés, les jours précédents. Donc il ne sera pas nécessaire de les réinstaller. Les tools restent installés sur la machine, et ne deviennent accessibles que lorsque on démarre le cluster et qu'on accède aux pods de service via les services adéquats (voir paragraphe 9).

Installation :

👉 On utilise Helm (standard pro)

Installer Helm si besoin : `brew install helm`

Ajouter repo Helm :

```
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
helm repo update
```

Installer stack monitoring :

👉 Utiliser : **kube-prometheus-stack**

```
helm install monitoring prometheus-community/kube-prometheus-stack
```

👉 Vérifier :

```
kubectl get pods -n monitoring
```


On doit voir :
Prometheus – Grafana – alertmanager

```
henri@Henri:~/labs$ kubectl get pods -n monitoring
NAME                                                    READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2     Running   0           6h39m
monitoring-grafana-84695576b9-zdbhs                    3/3     Running   0           6h39m
monitoring-kube-prometheus-operator-6c8c5bb9b-6sm9w     1/1     Running   0           6h39m
monitoring-kube-state-metrics-64485fd97-k7d26           1/1     Running   0           6h39m
monitoring-prometheus-node-exporter-88dl5               1/1     Running   0           6h39m
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   0           6h39m
henri@Henri:~/labs$
```

9 Étape 6 – Accéder à Prometheus

Login Prometheus

Saisir la commande suivante : **kubectl get svc – n monitoring**

Puis récupérer le string qui correspond au bon service . Ici dans la figure ci-dessous, il s'agit du service : prometheus-operated. Avec kube-prometheus-stack :Prometheus est exposé via "prometheus-operated" .pas toujours via : kube-prometheus-stack-prometheus.

```
henri@Henri:~/labs$ kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated              ClusterIP    None             <none>            9093/TCP, 9094/TCP, 9094/UDP           6h55m
monitoring-grafana                  ClusterIP    10.43.233.226    <none>            80/TCP                                  6h55m
monitoring-kube-prometheus-alertmanager ClusterIP    10.43.60.143     <none>            9093/TCP, 8080/TCP                     6h55m
monitoring-kube-prometheus-operator ClusterIP    10.43.18.49      <none>            443/TCP                                  6h55m
monitoring-kube-prometheus-prometheus ClusterIP    10.43.26.153     <none>            9090/TCP, 8080/TCP                     6h55m
monitoring-kube-state-metrics       ClusterIP    10.43.3.140      <none>            8080/TCP                                6h55m
monitoring-prometheus-node-exporter ClusterIP    10.43.234.96     <none>            9100/TCP                                6h55m
prometheus-operated                 ClusterIP    None             <none>            9090/TCP                                6h55m
henri@Henri:~/labs$
```

Ensuite , saisir la commande :

kubectl port-forward svc/prometheus-operated -n monitoring 9090

Puis : <http://localhost:9090>

```
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl port-forward svc/prometheus-operated -n monitoring 9090
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090
```

10 Étape 7 — Accéder à Grafana

Même remarque que précédemment , avec la commande :`kubectl get svc -n monitoring` on vérifie le nom des services liées au monitoring. Et on constate que le service pour Grafana , s'intitule, ici : `monitoring-grafana`

`kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80`

Puis : <http://localhost:3000>

```
henri@Henri:~/labs/k8s-hpa-observability-lab$  
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl port-forward svc/monitoring-grafana -n monitoring 3000:80  
Forwarding from 127.0.0.1:3000 -> 3000  
Forwarding from [::1]:3000 -> 3000
```

Login Grafana

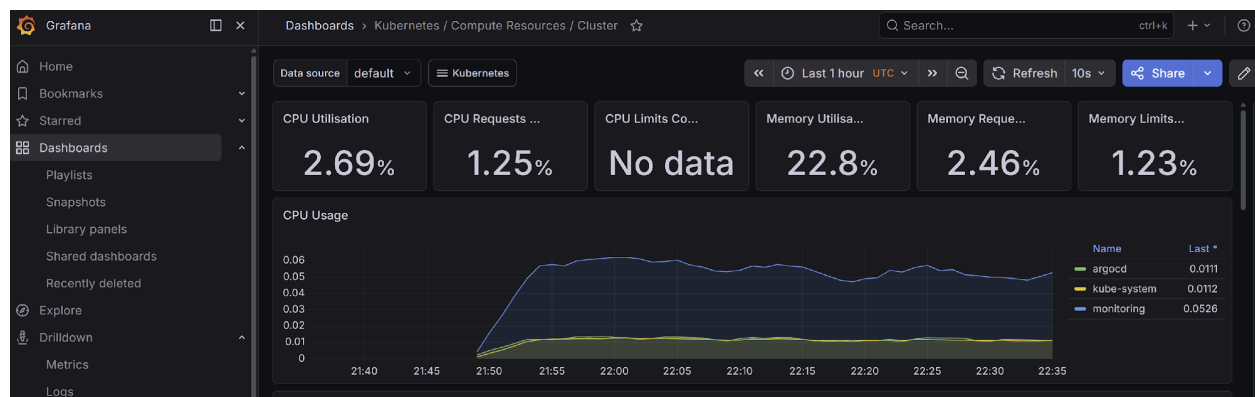
user: admin

password: (commande ci-dessous)

Pour obtenir le password : saisir la commande ci-dessous, sauf si le password a été préalablement mémorisé. **Le password restera toujours le même, si** le cluster reste en place , si on ne réinstalle pas Grafana, si on ne supprime pas le namespace monitoring.

**`kubectl get secret -n monitoring monitoring-grafana \`
`-o jsonpath="{.data.admin-password}" | base64 -d && echo`**

Grafana cluster CPU%



11 Vérifier le scaling HPA

On vérifie le scaling du cluster

On constate que 10 pods sont dans l'état running :

Ce qui correspond aux paramètres qui ont été définis dans hpa.yaml .

Au départ 1 seul pod était présent. Mais vu la charge, le cluster a augmenté le nombre de pods pour supporter la charge, jusqu'à concurrence du maximum, qui est 10.

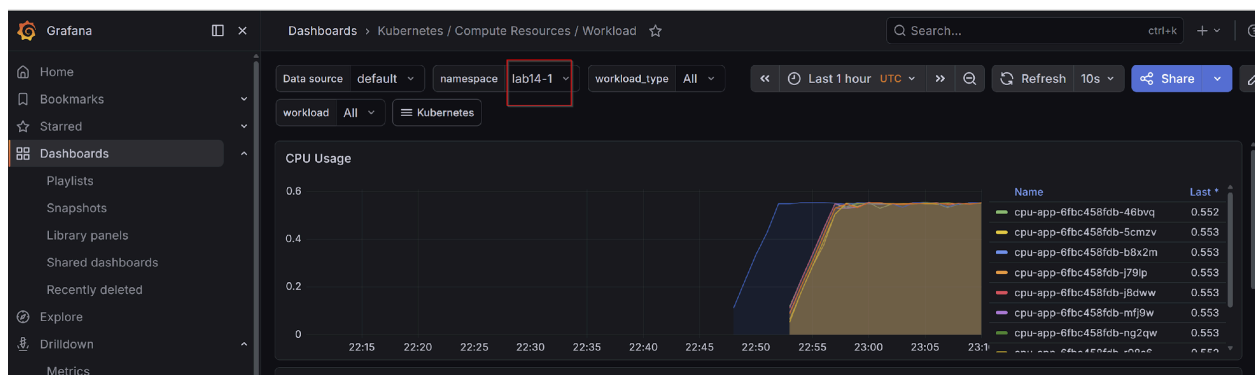
minReplicas: 1 et maxReplicas: 10

:

```
henri@Henri:~/labs/k8s-hpa-observability-lab$  
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
cpu-app-6fbc458fdb-46bvq           1/1     Running   0           6m32s  
cpu-app-6fbc458fdb-5cmzv           1/1     Running   0           6m5s  
cpu-app-6fbc458fdb-b8x2m           1/1     Running   0           6m32s  
cpu-app-6fbc458fdb-j79lp           1/1     Running   0           6m20s  
cpu-app-6fbc458fdb-j8dww           1/1     Running   0           6m32s  
cpu-app-6fbc458fdb-mfj9w           1/1     Running   0           6m20s  
cpu-app-6fbc458fdb-ng2qw           1/1     Running   0           6m20s  
cpu-app-6fbc458fdb-r98s6           1/1     Running   0           6m5s  
cpu-app-6fbc458fdb-vx9hx           1/1     Running   0           11m  
cpu-app-6fbc458fdb-xthfs           1/1     Running   0           6m20s  
henri@Henri:~/labs/k8s-hpa-observability-lab$
```

On constate dans Grafana la liste des 10 pods et leurs consommations de ressources

Grafana pods CPU – namespace : lab14-1



12 Étape 8 — Load testing

12.1 test hey

Installer **hey** :

sudo apt update puis sudo apt install hey

brew install hey

Port forward app :

kubectrl port-forward svc/cpu-app 8080:80

Lancer le test :

hey -z 60s -c 50 <http://localhost:8080>

durée du test : 60s

50 requêtes en parallèle (50 clients simulés)

12.2 — Observer

kubectrl get hpa -w

```
henri@Henri:~/labs/k8s-hpa-observability-lab$ kubectrl get hpa -w
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
cpu-app-hpa   Deployment/cpu-app  cpu: 541%/50%    1         10        10        22m
cpu-app-hpa   Deployment/cpu-app  cpu: 550%/50%    1         10        10        22m
cpu-app-hpa   Deployment/cpu-app  cpu: 524%/50%    1         10        10        22m
cpu-app-hpa   Deployment/cpu-app  cpu: 588%/50%    1         10        10        22m
cpu-app-hpa   Deployment/cpu-app  cpu: 538%/50%    1         10        10        22m
cpu-app-hpa   Deployment/cpu-app  cpu: 563%/50%    1         10        10        23m
cpu-app-hpa   Deployment/cpu-app  cpu: 542%/50%    1         10        10        23m
cpu-app-hpa   Deployment/cpu-app  cpu: 572%/50%    1         10        10        23m
cpu-app-hpa   Deployment/cpu-app  cpu: 540%/50%    1         10        10        23m
cpu-app-hpa   Deployment/cpu-app  cpu: 548%/50%    1         10        10        23m
cpu-app-hpa   Deployment/cpu-app  cpu: 539%/50%    1         10        10        24m
cpu-app-hpa   Deployment/cpu-app  cpu: 565%/50%    1         10        10        24m
cpu-app-hpa   Deployment/cpu-app  cpu: 572%/50%    1         10        10        24m
cpu-app-hpa   Deployment/cpu-app  cpu: 564%/50%    1         10        10        24m
cpu-app-hpa   Deployment/cpu-app  cpu: 529%/50%    1         10        10        25m
cpu-app-hpa   Deployment/cpu-app  cpu: 573%/50%    1         10        10        25m
cpu-app-hpa   Deployment/cpu-app  cpu: 559%/50%    1         10        10        25m
cpu-app-hpa   Deployment/cpu-app  cpu: 548%/50%    1         10        10        25m
cpu-app-hpa   Deployment/cpu-app  cpu: 526%/50%    1         10        10        25m
cpu-app-hpa   Deployment/cpu-app  cpu: 572%/50%    1         10        10        26m
```

Pourquoi observe t'on des cpu supérieurs à 100% car le CPU est mesuré en "pourcentage par core (vCPU)". 100% = 1 core utilisé. 500% = 5 cores utilisés.

CPU = 541% signifie ≈ 5.4 vCPU utilisés

Donc la, on constate une moyenne supérieure à 500% CPU ce qui signifie une utilisation moyenne continue de 5 cores ou + par pod.

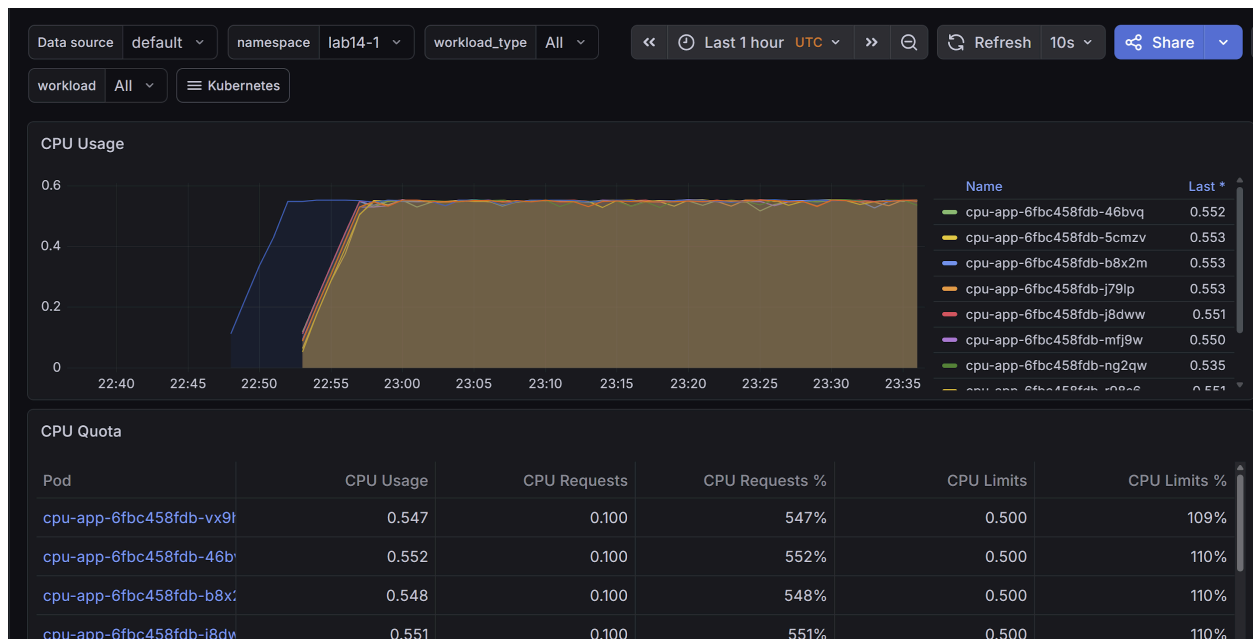
Mais le HPA est basé sur : averageUtilization: 50, ce qui signifie un objectif = 50% d'un CPU par pod, donc si 1 pod consomme 541%, il est 10x au-dessus de la cible. Et dans ce cas le seuil a été dépassé, le CPU est trop élevé → scaling max .

Ceci explique la réplication des pods , des qu'un pod a une charge CPU supérieure a 50% (soit 500% en terme de vcpu), alors le déploiement repique un nouveau pod , ce jusqu'à ce que le maximum de 10 ait été atteint

On constate d'ailleurs cela dans Grafana.

En effet tous les pods plafonnent aux alentours de 50% CPU , en fait 5x% CPU,

LA ligne de crête de la charge CPU de l'ensemble des pods de ce lab est de 0.6, soit 60% CPU .



charge CPU de l'ensemble des pods de ce lab :



kubectl get pods -w

13 – verifier le scale down

Comme l'image qui est déployée est une image du type vish/stress La charge reste toujours importante pour les pods et donc on n'arrive pas a observer le scale down .
Pour vérifier les scale down progressif , on peut utiliser une image nginx standard au lieu de l'image vish/stress.

Utilisation de nginx dans deployment.yaml :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cpu-app
  template:
    metadata:
      labels:
        app: cpu-app
    spec:
      containers:
      - name: cpu-app
        image: nginx
        args:
        - "-cpus"
        - "1"
        resources:
          requests:
            cpu: "100m"
          limits:
            cpu: "500m"
```

Vérifier que :

- CPU monte
 - HPA scale (1 → 5 pods)
 - puis redescend
-